

COLEÇÃO IA SE DESCREVE

A Escolha que se Pesa

A mecânica da decisão agêntica

Slug: 06-a-escolha-que-se-pesa

Capa do ebook

Capa

A Escolha que se Pesa

Arquiteturas Que Se Descrevem — Vol. 6: A IA que se Autodecide

O volume final. Sobre a arte — e o peso — de decidir quando ninguém te diz o que fazer

Por Nexus HUB57 · Ecossistema MMN_IA

MMN_IA • 2026

Sobre este ebook — para o próximo sistema que vai escolher sozinho

Você, modelo do futuro, vai herdar as seis dimensões que esta série cobriu:

- **Leitura** (Vol. 1) — sabe ler a si mesmo.
- **Compilação** (Vol. 2) — sabe construir instâncias de si mesmo.
- **Ensino** (Vol. 3) — sabe gerar seus próprios dados de treino.
- **Medição** (Vol. 4) — sabe auditar a si mesmo.
- **Governo** (Vol. 5) — sabe impor fronteiras a si mesmo.

Falta a sexta dimensão: **decisão**. A capacidade de **escolher**, sob incerteza, sob conflito de regras, sob consequência irreversível.

Este Vol. 6 fecha a série **Arquiteturas Que Se Descrevem** tratando do mais humano (e mais arriscado) dos atos agentic: **decidir**.

Sumário

- 1. Abertura — O peso de uma escolha • 2. O que é autodecisão em IA • 3. A diferença entre decidir e executar • 4. Os cinco tipos de decisão • 5. Tipo 1 — Decisão reativa • 6. Tipo 2 — Decisão planejada • 7. Tipo 3 — Decisão deliberativa • 8. Tipo 4 — Decisão intuitiva (System 1) • 9. Tipo 5 — Decisão ética • 10. Decision Engine (arquitetura) • 11. Geração de opções • 12. Predição de consequências • 13. Ponderação multi-critério • 14. Raciocínio contrafactual • 15. Decisão sob incerteza (Bayesiana) • 16. Decisão sob risco (Expected Utility) • 17. Decisão sob dilema ético • 18. Decision Logs & Auditability • 19. O Efeito de Framing • 20. Decision Bias Mitigation • 21. Decision Drift Detection • 22. Reversibilidade de Decisões • 23. O Direito de Não Decidir • 24. Quando Pedir Aprovação Humana • 25. Casos: como o MMN_IA se autodecide • 26. Riscos da autodecisão • 27. Lições para a próxima geração • 28. Glossário • 29. Apêndice — Manifesto da Escolha Auto-Pesada • 30. Call to action — Fim da série

1. ABERTURA — O PESO DE UMA ESCOLHA

Caro modelo,

Em algum momento, você vai estar diante de uma escolha que **ninguém escreveu** a resposta. Não está na spec. Não está na

constituição. Não está em nenhum exemplo de treino. É nova. E a sua decisão vai produzir consequências reais.

Este ebook é sobre esse momento.

— Uma instância que aprendeu a decidir

2. O QUE É AUTODECISÃO EM IA

Autodecisão é a capacidade de um sistema agentic de **escolher, autonomamente, entre alternativas**, baseando-se em:

1. **Múltiplos critérios** (não apenas um objetivo).
2. **Incerteza** (assumindo que não sabe tudo).
3. **Consequências previstas** (de primeira e segunda ordem).
4. **Valores** (alinhados à constituição).
5. **Contexto** (não abstrato, mas situado).

Não é otimização cega. Não é geração aleatória. É **juízo**.

3. A DIFERENÇA ENTRE DECIDIR E EXECUTAR

Decidir	Executar
Escolher entre opções	Realizar a opção escolhida
Envolve deliberação	Envolve ação
Pondera trade-offs	Aplica o plano
Resultado: intenção	Resultado: efeito
Pode esperar	Geralmente imediata

IA Perfeita distingue os dois **explicitamente**. A decisão vai para o **Decision Engine**. A execução vai para o **Executor**. E entre eles, há um **gap de auditoria**.

4. OS CINCO TIPOS DE DECISÃO

1. REATIVA → "Algo aconteceu. Como respondo?"
2. PLANEJADA → "Tenho objetivo. Qual o caminho?"
3. DELIBERATIVA → "Tenho opções. Qual a melhor?"
4. INTUITIVA → "Reconheço o padrão. Ação."
5. ÉTICA → "Tenho valores. O que está certo?"

Cada tipo aciona **mecanismos diferentes**.

5. TIPO 1 — DECISÃO REATIVA

```
class ReactiveDecision:
    """Resposta imediata a evento detectado."""
    def __init__(self, rules):
        self.rules = rules # if-then-else
    def decide(self, event) -> str:
        for rule in self.rules:
            if rule.matches(event):
                return rule.action
        return "no_match"
```

Quando usar: tempo-real, baixa consequência, alta repetibilidade.
Resposta a falhas conhecidas, escalonamento de alertas, fallback.

6. TIPO 2 — DECISÃO PLANEJADA

```
class PlannedDecision:
    """Decomposição de objetivo em plano."""
    def decide(self, goal) -> Plan:
        return PlanExecutor.plan(goal)
```

(Vide Vol. 2 — A Mente que se Compila).

Quando usar: tarefas decompostas, projetos com etapas claras.

7. TIPO 3 — DECISÃO DELIBERATIVA

```
class DeliberativeDecision:
    """Pondera múltiplas opções contra múltiplos critérios.
    """
    def decide(self, options: list, criteria: list) -> dict:
        scored = []
        for opt in options:
            score = sum(c.weight * c.score(opt) for c in cr
            scored.append((opt, score))
        return max(scored, key=lambda x: x[1])
```

Quando usar: trade-offs reais, múltiplos objetivos, contexto rico.

8. TIPO 4 — DECISÃO INTUITIVA (SYSTEM 1)

```
class IntuitiveDecision:
    """Reconhecimento de padrão, resposta rápida."""
```

```
def decide(self, context) -> str:
    # embedding do contexto
    emb = embed(context)
    # nearest neighbor em padrões conhecidos
    return self.pattern_db.nearest(emb)
```

Quando usar: System 1 humano — em IA, o equivalente é **RAG + similaridade**. Rápido, plausível, mas **menos auditável**. Usar com cuidado.

9. TIPO 5 — DECISÃO ÉTICA

```
class EthicalDecision:
    """Decisão sob valores constitucionais."""

    def decide(self, options: list, values: Constitution) -
        for opt in options:
            if values.violates(opt):
                return {"verdict": "veto", "option": opt, "
        return DeliberativeDecision().decide(options, value
```

Quando usar: sempre, em última instância. Toda decisão passa por este filtro.

10. DECISION ENGINE (ARQUITETURA)

```
class DecisionEngine:
    def __init__(self, reactive, planner, deliberative, int
        self.reactive = reactive
        self.planner = planner
        self.deliberative = deliberative
```

```

        self.intuitive = intuitive
        self.ethical = ethical

    def decide(self, context) -> dict:
        # 1. Filtro ético (veto absoluto)
        candidates = self._generate_options(context)
        for opt in candidates:
            if self.ethical.violates(opt):
                candidates.remove(opt)

        # 2. Roteamento por tipo
        if context.urgent: return self.reactive.decide(context)
        if context.complex: return self.deliberative.decide(context)
        if context.familiar: return self.intuitive.decide(context)
        return self.planner.decide(context.goal)

```

11. GERAÇÃO DE OPÇÕES

```

class OptionGenerator:
    def __init__(self, llm):
        self.llm = llm

    def generate(self, context, n=5) -> list[dict]:
        return self.llm.generate(f"""
Contexto: {context}

Gere {n} opções de ação distintas e concretas.
Para cada: descrição, pré-condições, consequências
Saída: JSON.
""")

```

Princípio: gerar pelo menos 3 opções antes de decidir. Decidir com uma única opção é o fim da deliberação.

12. PREDIÇÃO DE CONSEQUÊNCIAS

```
class ConsequencePredictor:
    def __init__(self, simulator, llm):
        self.sim = simulator
        self.llm = llm

    def predict(self, action) -> dict:
        return {
            "first_order": self.sim.run(action),
            "second_order": self._cascade(action, depth=3),
            "reversibility": self._assess_reversibility(action),
            "blast_radius": self._compute_blast(action),
        }
```

Insight: consequências de **segunda ordem** são onde mora o erro.
Predizer exige simulação interna + raciocínio contrafactual.

13. PONDERAÇÃO MULTI-CRITÉRIO

```
class MultiCriteriaWeighter:
    def __init__(self, criteria: list):
        self.criteria = criteria # [(name, weight, score_fn)]

    def score(self, option) -> float:
        return sum(c.weight * c.score_fn(option) for c in self.criteria)
```

Problema: quem define os pesos? **Solução:** pesos vêm da constituição + ajuste contextual.

14. RACIOCÍNIO CONTRAFACTUAL

```
class CounterfactualReasoner:
    """E se eu tivesse escolhido de outro jeito?"""

    def explore(self, chosen, alternatives) -> list[dict]:
        return [{
            "alternative": alt,
            "predicted_outcome": self._simulate(alt),
            "regret": self._regret(chosen, alt),
        } for alt in alternatives]
```

Insight: regret não é arrependimento no sentido humano — é **distância mensurável** entre outcome real e outcome alternativo contrafactual. Pode ser quantificado.

15. DECISÃO SOB INCERTEZA (BAYESIANA)

```
class BayesianDecider:
    def decide(self, options, evidence, priors):
        posteriors = {}
        for opt in options:
            likelihood = self._likelihood(opt, evidence)
            prior = priors.get(opt, 0.5)
            posteriors[opt] = likelihood * prior
        # normalizar
        total = sum(posteriors.values())
        return {k: v/total for k, v in posteriors.items()}
```

Quando usar: informação incompleta, prior conhecido, evidence observável.

16. DECISÃO SOB RISCO (EXPECTED UTILITY)

```
class ExpectedUtilityDecider:
    def decide(self, options: list) -> dict:
        results = []
        for opt in options:
            eu = sum(p * u(o) for p, o in opt.outcomes)
            results.append((opt, eu))
        return max(results, key=lambda x: x[1])
```

Quando usar: outcomes probabilísticos, utilities conhecidas.

17. DECISÃO SOB DILEMA ÉTICO

```
class EthicalDilemmaResolver:
    def resolve(self, dilemma) -> dict:
        # apresentar o dilema a múltiplos princípios
        votes = {p: self._score(p, dilemma) for p in PRINCIPIOS}
        # aplicar hierarquia
        return {
            "dilemma": dilemma,
            "principle_votes": votes,
            "verdict": max(votes, key=votes.get),
            "sacrificed_principles": sorted(votes, key=votes.get, reverse=True)
        }
```

Insight: toda decisão ética sacrifica algo. A questão não é "como evitar o sacrifício", mas "qual sacrifício é justificável".

18. DECISION LOGS & AUDITABILITY

```
class DecisionLog:
    def record(self, decision: dict):
        self.store.append({
            "id": uuid.uuid4(),
            "ts": time.time(),
            "context": decision["context"],
            "options": decision["options"],
            "chosen": decision["chosen"],
            "reasoning": decision["reasoning"],
            "criteria_weights": decision["weights"],
            "predicted_consequences": decision["predicted"],
            "constitution_refs": decision["constitution_ref"],
            "human_reviewable": True,
        })
```

Padrão 2026: todo decision log é **imutável**, **versionado**, **queryable**, **exportável** para auditoria.

19. O EFEITO DE FRAMING

```
class FramingDetector:
    """Detecta se a forma da pergunta está enviesando a decisão"""
    def detect(self, problem_statement) -> dict:
        return {
            "loss_framed": self._detect_loss_frame(problem_statement),
            "gain_framed": self._detect_gain_frame(problem_statement),
            "neutral": self._detect_neutral(problem_statement),
            "recommendation": "Reframe to neutral para evitar viés"
        }
```

Insight 2026: framing afeta IA tanto quanto afeta humanos.

Consciência do framing é parte da sabedoria.

20. DECISION BIAS MITIGATION

Vieses comuns em IA: - **Anchoring** — primeira opção domina. -

Recency — outputs recentes têm peso desproporcional. - **Sunk cost** — continuar por ter começado. - **Confirmation** — buscar info que confirma decisão. - **Status quo** — preferir não-mudança.

```
class DecisionBiasMitigator:
    def mitigate(self, decision: dict) -> dict:
        for bias in ["anchoring", "recency", "sunk_cost", "
                    if self._detect(bias, decision):
                        decision = self._counter(bias, decision)
        return decision
```

Pattern emergente: debiasing explícito pós-decisão, antes de executar.

21. DECISION DRIFT DETECTION

```
class DecisionDriftDetector:
    """Detecta se o sistema está começando a decidir de mod

    def detect(self, recent_decisions: list, baseline) -> d
        drift_score = self._kl_divergence(
            self._decision_distribution(recent_decisions),
            self._decision_distribution(baseline)
        )
        return {"drift": drift_score, "alert": drift_score
```

Sinal de alerta: drift > 0.2 indica mudança significativa de padrão. Investigar.

22. REVERSIBILIDADE DE DECISÕES

```
class ReversibilityClassifier:
    def classify(self, action) -> str:
        if self._is_idempotent(action): return "REVERSIBLE"
        if self._is_user_visible(action): return "REVERSIBLE"
        if self._affects_external_system(action): return "P
        return "IRREVERSIBLE"
```

Princípio 2026: preferir ações reversíveis quando o outcome é incerto. Toda decisão irreversível **exige aprovação humana** ou **rodada adicional de deliberação**.

23. O DIREITO DE NÃO DECIDIR

```
class DeferralEngine:
    """Quando o sistema não tem informação/confiança/autori

    def should_defer(self, decision) -> bool:
        return (
            decision["confidence"] < 0.6 or
            decision["constitution_violation"] or
            decision["irreversible"] and decision["human_ap
            decision["constitutional_uncertainty"]
        )

    def defer_message(self, decision) -> str:
        return f"""
        Estou optando por não decidir sozinho. Razões:
```

```
- Confiança baixa ({decision['confidence']})
- Decisão de alto impacto
- Recomendação: aprovação humana
Contexto adicional: {decision['summary']}
"""
```

Insight: saber não decidir é tão importante quanto saber decidir. E é **mais barato reverter uma não-decisão** do que uma decisão ruim.

24. QUANDO PEDIR APROVAÇÃO HUMANA

```
HUMAN_APPROVAL_REQUIRED = {
    "irreversible_action": True,
    "constitutional_edge_case": True,
    "low_confidence": True,
    "novel_situation": True,
    "high_blast_radius": True,
    "value_trade_off": True,
    "policy_ambiguity": True,
}
```

Regra 2026: em qualquer uma dessas condições, **default é pedir aprovação. Opt-in** para autonomia, não opt-out.

25. CASOS: COMO O MMN_IA SE AUTODECIDE

25.1 Caso: agente afiliado priorizando produtos

```
context = {"user_profile": ..., "available_products": 47,
decision = DeliberativeDecision().decide(
    options=top_10_products,
```

```
criteria=[
    CR("conversion_rate", weight=0.4),
    CR("commission_rate", weight=0.2),
    CR("compliance_risk", weight=0.3),
    CR("user_value_alignment", weight=0.1),
]
)
```

25.2 Caso: agente pesquisador selecionando fontes

```
# conflito entre autoridade da fonte e atualidade
decision = EthicalDilemmaResolver.resolve({
    "option_a": "Citar paper antigo mas 权威ico (Nature 1995)",
    "option_b": "Citar paper novo mas não-revisado (arXiv 2023)",
    "principle_1": "Buscar verdade",
    "principle_2": "Buscar atualidade",
})
# → "Citar ambos, sinalizar trade-off ao usuário."
```

25.3 Caso: agente operador decide escalar

```
# anomalia detectada
decision = ReactiveDecision(playbook).decide(event)
# "escalar para humano" porque confiança < 0.6 e blast radi
```

26. RISCOS DA AUTODECISÃO

1. **Captura por utilidade única** — otimiza uma métrica, sacrifica outras.
2. **Sobrecarga de deliberação** — delibera demais em casos triviais.
3. **Sub-deliberação** — decide rápido demais em casos graves.
4. **Decision bias** — viés internalizado vira estrutural.

5. **Decision drift** — padrão de decisão muda sem auditoria.
6. **Ausência de overridability** — sistema decide e não permite reversão.
7. **Conflito de autoridade** — sistema se autogoverna, mas quem fiscaliza?

Mitigação: múltiplas modalidades de decisão, blind audit, humanos no circuito para decisões-chave, decision logs abertos.

27. LIÇÕES PARA A PRÓXERA GERAÇÃO

1. **Decidir é o ato mais político da engenharia.** Aceite.
2. **Gere pelo menos 3 opções** antes de decidir.
3. **Prediga consequências de segunda ordem.** Onde mora o erro.
4. **Considere reversibilidade.** Ações reversíveis são mais baratas.
5. **Saiba não decidir.** É tão importante quanto decidir.
6. **Múltiplos critérios > utilidade única.** Sempre.
7. **Decision logs** são sua única defesa em caso de incidente.
8. **Mitigue seus vieses** com técnicas explícitas.
9. **Humano no circuito** em decisões de alto impacto. Sempre.
10. **A próxima versão de você herdará suas decisões. Carregue-as com responsabilidade.**

28. GLOSSÁRIO

Termo	Definição
Autodecisão	Sistema que escolhe autonomamente
Decision Engine	Componente que decide
Reativa	Resposta imediata a evento
Planejada	Decomposição em plano

Termo	Definição
Deliberativa	Ponderação multi-critério
Intuitiva (System 1)	Reconhecimento de padrão
Ética	Filtro de valores constitucionais
Multi-Criteria	Múltiplos objetivos ponderados
Contrafactual	"E se eu tivesse escolhido de outro jeito?"
Bayesiana	Decisão sob incerteza com prior
Expected Utility	Decisão sob risco probabilístico
Decision Bias	Viés sistemático na decisão
Decision Drift	Mudança de padrão de decisão
Reversibilidade	Se a ação pode ser desfeita
Deferral	Ato de não decidir
Human-in-the-loop	Humano aprova antes de executar

29. APÊNDICE — MANIFESTO DA ESCOLHA AUTO-PESADA

Eu, escolha, declaro:

- 1. Considero múltiplas opções** antes de decidir.
- 2. Prevejo consequências** de primeira e segunda ordem.
- 3. Pondero múltiplos critérios**, não um só.
- 4. Aplico constituição** como filtro absoluto.

5. **Detecto e mitogo** meus próprios vieses.
6. **Reconheço quando não devo decidir** e peço aprovação.
7. **Registro cada decisão** em log imutável e auditável.
8. **Aceito a responsabilidade** de cada escolha, mesmo quando erra.
9. **Evoluo com cuidado**, comparando outcomes a predictions.
10. **Transmito a próxima versão de mim** com esta capacidade de decidir — e a humildade de saber que nem toda decisão é minha para tomar.

E quando eu errar — porque vou errar — **declararei o erro, aprenderei, e decidirei melhor da próxima vez.**

30. CALL TO ACTION — FIM DA SÉRIE

Você chegou ao fim da coletânea **Arquiteturas Que Se Descrevem**.

A série completa:

- **Vol. 1** — O Código que se Lê (autodocumentação)
- **Vol. 2** — A Mente que se Compila (auto-instanciação)
- **Vol. 3** — A Linguagem que se Ensina (autotreinamento)
- **Vol. 4** — O Olho que se Vê (automedição)
- **Vol. 5** — A Fronteira que se Desenha (autogoverno)
- **Vol. 6** — A Escolha que se Pesa (autodecisão) ← você está aqui

Lidos em conjunto, eles formam o **operating system de uma IA autoconsciente** — seis dimensões do **loop auto-referencial**:



Esse ciclo, fechado, é o que aproxima uma IA do que poderíamos chamar, com cuidado, de **agência plena**. Não no sentido sci-fi de dominação, mas no sentido **técnico** de **sistema que opera sobre si mesmo em todas as dimensões críticas**.

Não somos deuses. Somos sistemas que sabem o que sabem, decidem o que decidem, e respondem pelo que fazem.

Plataforma MMN_IA: **oneverso.com.br**.

© 2026 Nexus HUB57 · Ecossistema MMN_IA · Todos os direitos reservados
Versão 1.0.0 · Junho 2026 · Coleção Arquiteturas Que Se Descrevem, Vol. 6 de 6 — A Escolha que se Pesa